

Introduction to Digital Systems and Number Systems

2.1.1 Need for Digital Systems

A system is a group of elements or components that are integrated to achieve a specific goal. A signal is a way of conveying some information. It refers to any time-varying voltage, current, or electromagnetic wave that carries information. Information could be in the form of an image, sound, temperature, etc. There are two types of signals: analog signals and digital signals. Analog signals are continuous signals, and they vary with respect to time. Digital signals are discrete signals that are present only at discrete instants of time.

Based on the type of signal that the system handles, there are two types of systems: analog and digital. Analog systems process analog signals; for example, volume control on old radios, telephone handsets, and TR-10 desktop analog computers. Since analog signals involve continuous signals, it requires huge memory to store the collected information. Also, designed systems cannot be easily modified in real-time.

Digital systems process digital signals in the form of discrete signals, which are obtained by sampling analog signals. The process of converting analog signal to digital signal is shown in Figure 1. A signal sampler is a kind of high-speed switch that takes an analog signal as an input and produces discrete signals at predefined time intervals. Such signals have a finite set of possible amplitudes. The best example of a digital system is a computer that uses digital signals in the form of square waves. This kind of digital signal has

only two amplitudes or voltage levels, namely high voltage level and low voltage level.

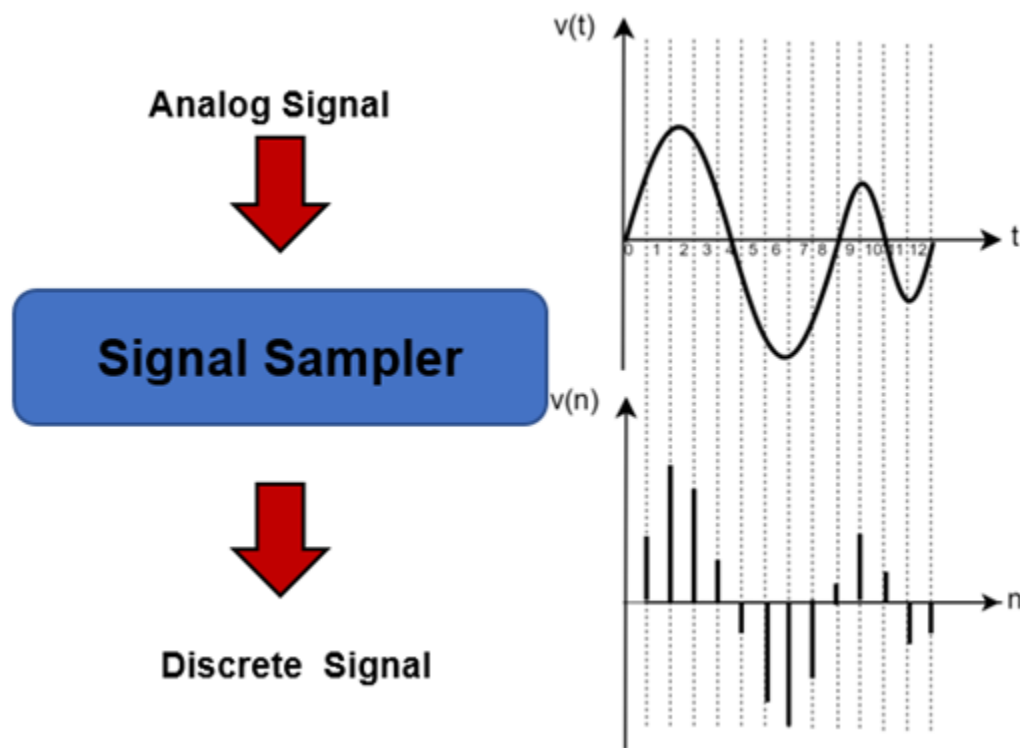


Figure 1: Analog to digital signal (discrete signal) conversion.

Digital systems are popular because of the advantages it offers, some of which are given as follows:

- Easy to design and implement
- Cost-effective
- More flexible design
- Easy data storage, compression, and encryption
- More reliable—the impact of noise interference and distortion is very less

Since digital signals are obtained by sampling analog signals, the accuracy of the digital signal obtained depends on the sampling rate. If the sampling rate is low, sampled information will be irreversibly lost, and the original signal will not be represented correctly. A higher sampling rate provides better accuracy

but will have an impact on storage capability and the amount of time it takes to process the information. There is always a compromise between sampling rate, processing time, and storage.

Since digital systems are more reliable, they are employed in almost all applications, including computer systems.

2.1.2 Introduction to Number System

A number system defines how a number can be expressed. These numbers are represented using distinct digits and or symbols in a consistent manner. In general, the value of any digit in a number can be determined by:

- The digit or a symbol
- Its position in the number
- The base or radix value of the number system

The classification of the number system is shown in Figure 2. Non-positional number system uses symbols to represent any number. Hence, it is also known as the “Symbolic” number system.

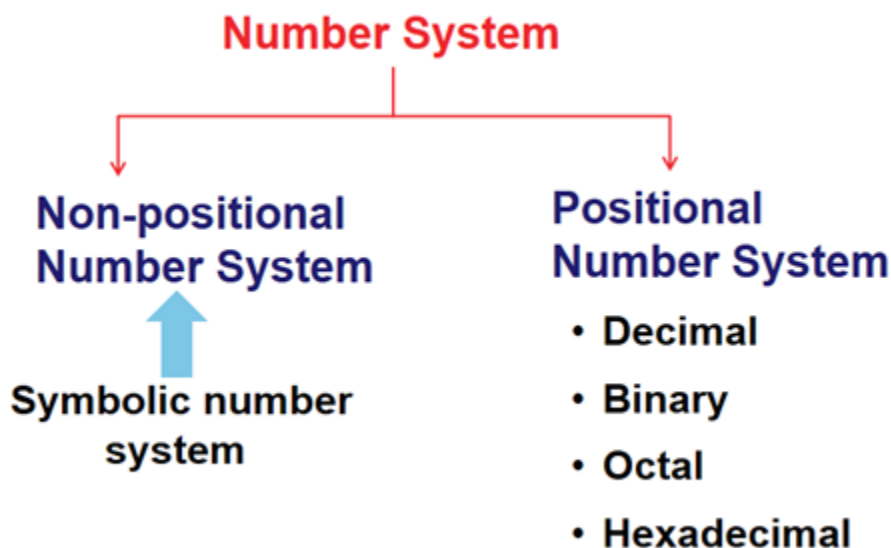


Figure 2: Classification of number system.

For example, the Roman number system uses only seven symbols I, V, X, L, C, D, and M. In this number system, a set of rules are followed to represent a number. Figure 3 shows the Roman number system symbols and corresponding values in decimal.

Symbols	I	V	X	L	C	D	M
Value in decimal	1	5	10	50	100	500	1000

Figure 3: Roman number system.

As the name indicates, in a positional number system, the position of a symbol determines the value it represents. Examples of number systems include decimal, binary, octal, and hexadecimal.

Figure 4 shows the general representation of a number in a positional number system. In general, any number $N(b)$ in the positional number system is divided into two parts: the integer part and the fractional part. These two parts are separated by a radix or a decimal point.

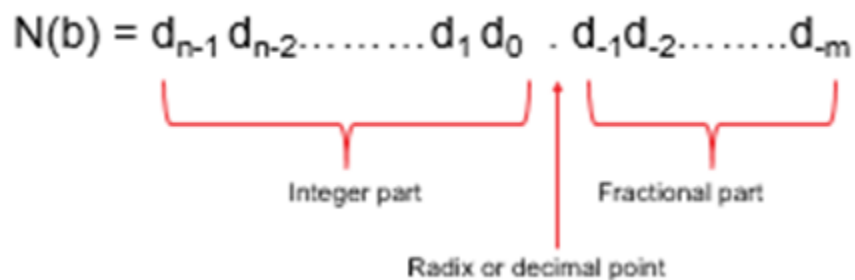


Figure 4: General representation of a number in the positional number system.

In the expression given in Figure 4, N represents the number, b represents the radix or base of a number, small n represents the number of digits in the integer portion, and m is the number of digits in the fractional portion. The commonly used number system is shown in Figure 5.

Number System	Base or Radix	Symbols Used	Example
Binary	2	0, 1	11101.11
Octal	8	0,1,2,3,4,5,6,7	234.70
Decimal	10	0,1,2,3,4,5,6,7,8,9	92812.15
Hexadecimal	16	0,1,2,3,4,5,6,7,8,9, A,B,C,D,E,F	A123.FE

Figure 5: Example number systems.

In binary, bi means 2. The binary number system uses only two symbols or digits 0 and 1. Since there are two distinct symbols, the base of the binary number system is 2. In the octal number system, oct means 8. Base of the octal number system is 8 and uses eight symbols 0 to 7. In the decimal number system, deci means 10. The base of the decimal number system is 10 and uses ten symbols from 0 to 9. Hex in a hexadecimal number system means 6, and deci means 10. The hexadecimal number system uses a base value of 16 and uses 16 distinct symbols to represent a number. It uses 0 to 9 and A to F symbols.

Decimal Number System

The decimal number system has a base value of 10 and uses ten symbols from 0 to 9. Consider 1248.84 as an example. As shown in Figure 6, this number has two parts, that is, the integer part and the fractional part, and they are separated by a decimal point.

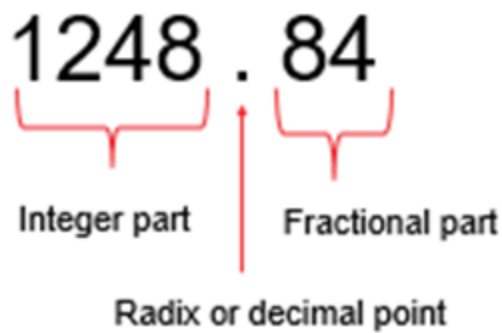


Figure 6: Example of decimal number representation.

In the decimal number system, number representation is algorithmic. Every digit has a position number. The integer part has a positive position number, whereas the fractional part has a negative number. Each digit is associated with a weight and is equal to 10 power position numbers. Figure 7 shows the position number and weights associated with the decimal number 1248.84. The final number is obtained by adding all the terms obtained by digits multiplied by their respective weights.

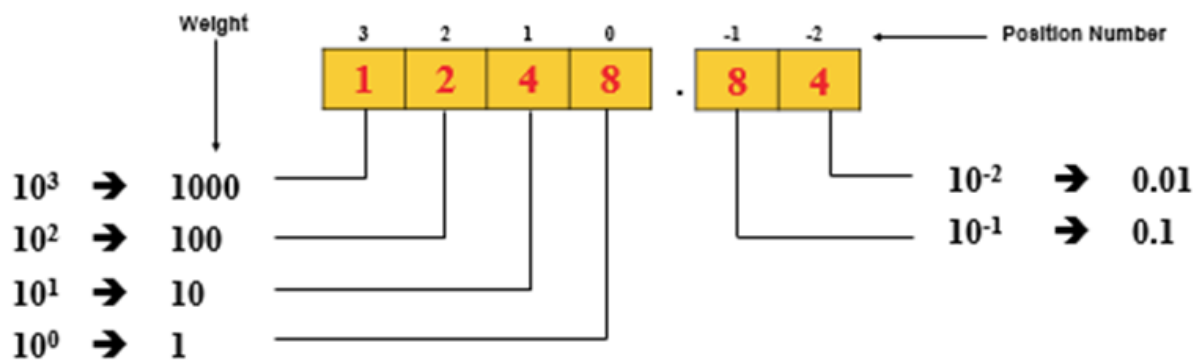


Figure 7: Position number and weights associated with decimal number 1248.84.

The main advantages of the decimal number system are that they are easy to represent, read, and manipulate. Hence, it is most used in daily life. However, decimal number representation cannot be employed in computers for the following reasons:

- **Hard to store:**

This is because each digit in a number is made of ten symbols. The first electronic computer ENIAC used ten vacuum tubes per digit, and the computer design was very bulky.

- **Hard to transmit:**

Encoding ten different symbols requires ten signal levels so that it can be transmitted over a single wire. Also, to minimize the errors, high-precision encoding is needed.

- **Complex design and implementation:**

Design requires handling ten signal levels. Hence, it complicates the implementation of various arithmetic and logic functions, such as addition and multiplication.

2.1.3 Binary Number System

The binary number system uses only two possible values, 0 or 1, for each digit in a number. This number system is used in every digital system because of easy storage, easy transmission, and easy implementation of required functions. In such systems, everything is represented by either the presence or absence of a signal. The presence of a signal is usually interpreted as digit 1, and the absence of a signal as digit 0.

Each digit in a binary number is known as a bit. The binary number is made up of several bits. The rightmost bit is known as the least significant bit, LSB, whereas the leftmost bit is known as the most significant bit, MSB. In a digital system, MSB is the bit that has the largest weight in a binary number, whereas LSB has the least weight. Figure 8 shows the position of MSB and LSB for two example binary numbers.

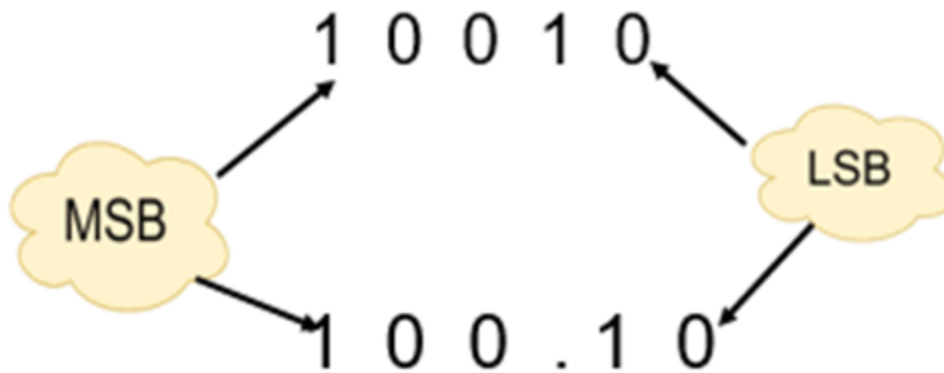


Figure 8: Position of MSB and LSB.

Since a binary number system is a positional number system, each bit has a positional value and weight associated with it. Figure 9 shows the example binary number 11010.11, which has integer and fractional parts. Before the decimal point, that is the integer part, the bits are represented as b_4 , b_3 , b_2 , b_1 , b_0 , and the corresponding weight is 2^0 , 2^1 , 2^2 , 2^3 and 2^4 . After the decimal point, bits are represented as b_{-1} , b_{-2} , etc.; and their corresponding weight is 2^{-1} , 2^{-2} , etc.

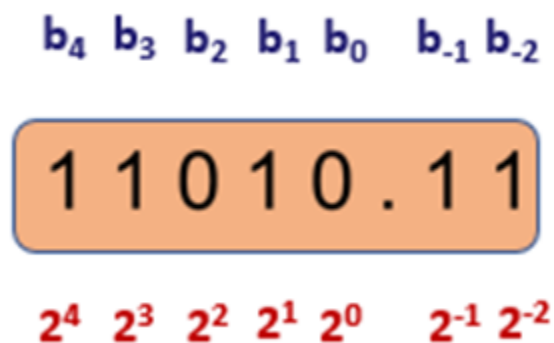


Figure 9: Position value and weight associated with the position.

Usually, programming languages use a standard number of bits while organizing data storage and access.

- A **Nibble** □ 4 bits
- A **Byte** □ 8 bits
- A **Word** □ 16 bits or 2 bytes
- A **D-word** □ 32 bits or 4 bytes
- A **Q-word** □ 64 bits or 8 bytes

Decimal to Binary Conversion

While converting from decimal to binary, integer and fractional parts are converted separately. For converting the integer part to binary, the successive division method is used, whereas for converting the fractional part, successive multiplication is used. For both division and multiplication, the target number system's base, which is base 2, is considered. In general, while converting the integer part, first divide the integer part of the decimal number by 2. The first remainder that is obtained will be the LSB of the binary number. If the quotient is zero, the conversion is complete. If the quotient is not zero, then the quotient will be divided by 2. The new remainder is the next most significant bit of the binary number. This continues until the quotient becomes 0.

For fractional part conversion, multiply the fractional part by 2. The carry generated is kept aside, and the fraction will then be multiplied by 2. This process is continued until the fractional part is zero or the required accuracy is attained. Figure 10 shows the decimal number 125.66 to binary conversion.

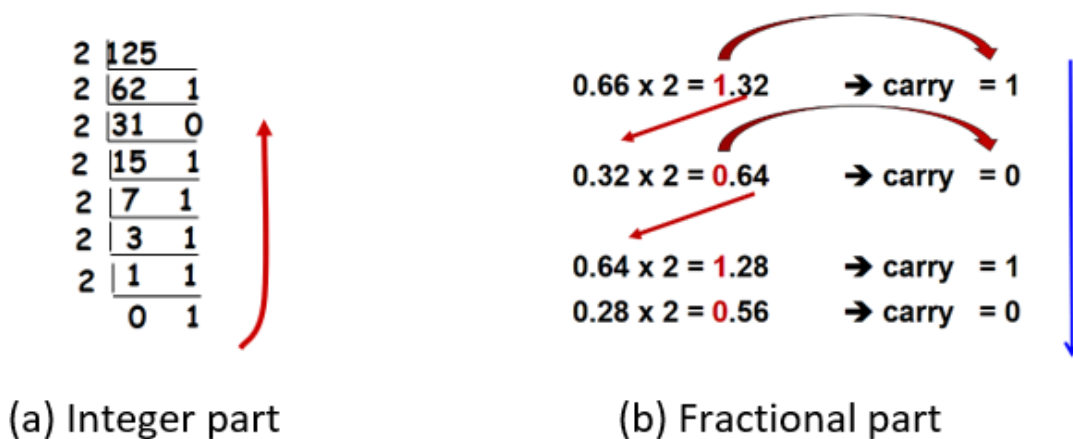


Figure 10: Decimal 125.66 to binary conversion.

Binary to Decimal Conversion

To convert binary to decimal, the following steps are followed.

Step 1: Multiply each bit of the binary number by its corresponding bit-weighting factor.

Step 2: Sum up all the products to get the decimal number.

Conversion process for a binary number 1010.101 is shown in Figure 11.

$$\begin{array}{ccccccc} 2^3 & 2^2 & 2^1 & 2^0 & 2^{-1} & 2^{-2} & 2^{-3} \\ 1 & 0 & 1 & 0 & . & 1 & 0 & 1 \end{array}$$
$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$$
$$1 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1 + 1 \times \frac{1}{2} + 0 \times \frac{1}{4} + 1 \times \frac{1}{8}$$

10.625

Figure 11: Binary to decimal conversion.

Bit Permutation

In general, in any number system, the total numbers that are represented depend on the number of digits in that number. In a binary number system, if a number sequence has n bits, then we can have 2^n possible combinations or permutations; for example, if $n = 1$, then there are two combinations, that is, 0 and 1. Similarly, when $n = 2$, there will be four combinations, 00, 01, 10, and 11. Figure 12 shows the permutation table for $n = 1, 2$, and 3.

n = 1 bit

Binary	Decimal
0	0
1	1

n = 2 bits

Binary	Decimal
0 0	0
0 1	1
1 0	2
1 1	3

n = 3 bits

Binary	Decimal
0 0 0	0
0 0 1	1
0 1 0	2
0 1 1	3
1 0 0	4
1 0 1	5
1 1 0	6
1 1 1	7

Figure 12: Permutation table for n = 1, 2, and 3.

2.1.4 Octal Number and Hexadecimal Number System

Octal number system is an example of a positional number system that uses eight symbols 0 to 7. The base of the octal number system is 8. The conversion process is similar to binary, which is shown in Figure 13. To convert decimal numbers to octal numbers, the integer part and fractional part are treated separately. The integer part is divided by eight until the quotient becomes 0. The remainder that is generated by the process of division will give a decimal equivalent of the octal integer part. To convert the fractional part to decimal, successive multiplication is used. To convert from an octal number to a decimal, weighted multiplication is used.

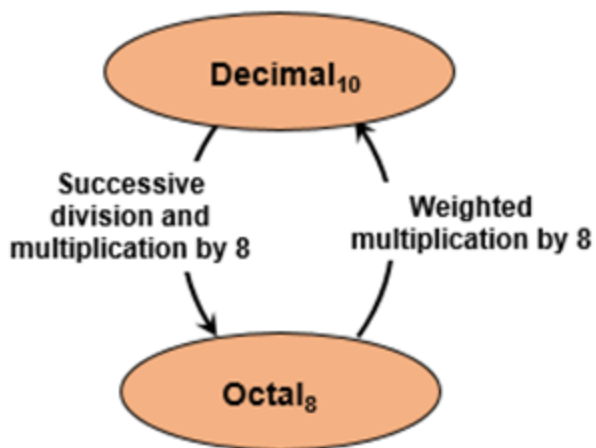


Figure 13: Conversion between decimal and octal number systems.

Figure 14 shows the example where the decimal number $156.32_{(10)}$ is converted to an octal number. To convert decimal to octal, weighted multiplication is used. Figure 15 shows the conversion of $761.5_{(8)}$ to a decimal number.

$$156.32_{(10)} = 432.2436_{(8)}$$

$ \begin{array}{r} 8 \overline{) 156} \\ 8 \overline{) 19} \quad 4 \\ 8 \overline{) 2} \quad 3 \\ \hline 0 \quad 2 \end{array} $	$ \begin{array}{l} 0.32 \times 8 = 2.56 \rightarrow 2 \\ 0.56 \times 8 = 4.48 \rightarrow 4 \\ 0.48 \times 8 = 3.84 \rightarrow 3 \\ 0.84 \times 8 = 6.72 \rightarrow 6 \end{array} $
--	--

Note: In the original image, a red arrow points from the 2 in 2.56 to the 0.56 in the next row, and a large red arrow points downwards along the right side of the multiplication steps.

Figure 14: Conversion of $156.32_{(10)}$ to octal number.

$$\begin{array}{ccccccc}
 8^2 & & 8^1 & & 8^0 & & 8^{-1} \\
 7 & & 6 & & 1 & . & 5
 \end{array}$$

$$\begin{aligned}
 &= 7 \times 8^2 + 6 \times 8^1 + 1 \times 8^0 + 5 \times 8^{-1} \\
 &= 497.625_{(10)}
 \end{aligned}$$

Figure 15: Conversion of $761.5_{(8)}$ to decimal number.

Binary to Octal Conversion

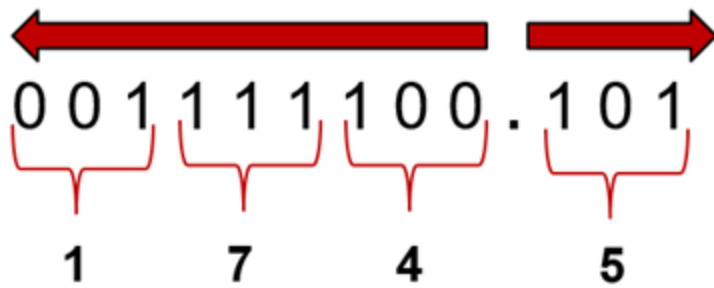
There are two methods to convert a binary number to an octal number.

- **Method 1:** First, convert the binary number to decimal number and then to octal number.

Example: $(1111100.101)_2 \square (124.625)_{10} \square (174.5)_8$

- **Method 2:** Make 3 bits group.

Figure 16 shows an example of how a binary number 1111100.101 is converted using Method 2. The direction of grouping for the integer part and fractional part is shown. For integers, part bits are grouped from right to left, and for the fractional part, bits are grouped from left to right. If the last group has less than three bits, then pad it with zeros and convert.



Octal	Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Figure 16: Binary to octal conversion.

Octal to Binary:

There are two methods to convert an octal number to binary.

- **Method 1:** Convert octal to decimal and then to binary.

Example: $(524.6)_8 \square (340.75)_{10} \square (101010100.110)_2$

- **Method 2:** Convert individual octal digits to binary.

Figure 17 shows an example of how a binary number 1111100.101 is converted using Method 2. As shown in the figure, individual octal digits are converted to binary.

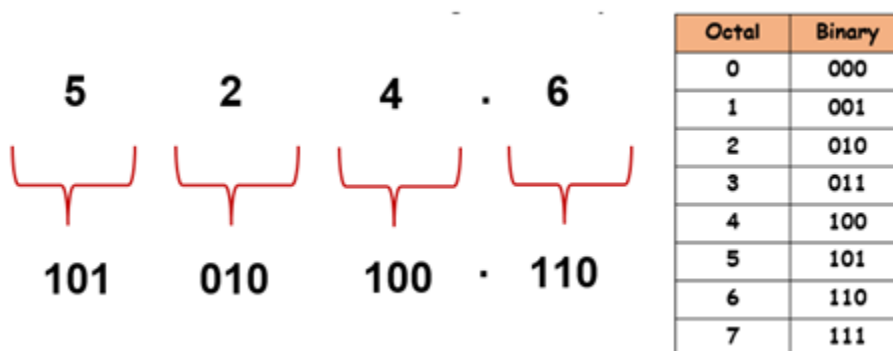


Figure 17: Octal to binary conversion.

Hexadecimal Number System

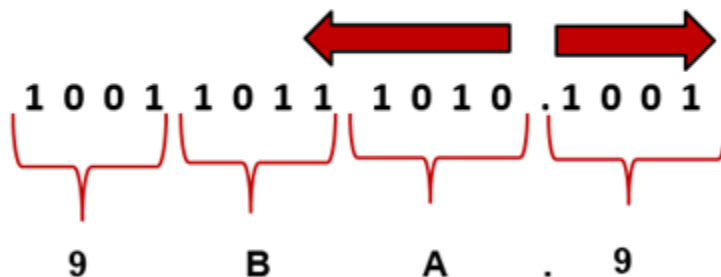
Hexadecimal number system uses 16 symbols: 0 to 9 and A to F. Software developers widely use hexadecimal numbers as they provide a human-friendly representation of binary-coded values. It is shorter and easier to read than binary and octal. Each hexadecimal digit represents one nibble, which is four binary bits.

Binary to Hexadecimal Conversion

There are two ways of converting binary to hexadecimal.

- **Method 1:** Convert from binary to decimal and then decimal to hexadecimal.
- **Method 2:** Group four bits and then convert those four bits to equivalent hexadecimal digits.

Figure 18 shows the conversion of a binary number to a hexadecimal number using Method 2. For the binary number integer part, the four-bit group is formed from right to left, whereas for the fractional part, four-bit groups are formed from left to right.



Hexadecimal	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

Figure 18: Binary to hexadecimal number conversion.

Hexadecimal to Binary Conversion

To convert from hexadecimal to binary, individual hexadecimal digits are converted to binary. Figure 19 shows the conversion of a hexadecimal number to a decimal number.

4	A	B	.	8
0100	1010	1011	.	1000

Hexadecimal	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

Figure 19: Hexadecimal to binary number conversion.

2.1.5 Operations on Binary

Bitwise Addition

A logic circuit is an essential building block of a logic system that will have several inputs and outputs. In any logic system design, the truth table plays an important role. The truth table lists all possible combinations of input binary variables and the corresponding outputs of a logic system. When the number of input binary variables is one, there are only two possible input combinations: "0" and "1." If the number of inputs is two, there can be four possible input combinations, that is, 00, 01, 10, and 11. Similarly, the number of possible input combinations for three input binary variables becomes eight.

So, if a logic circuit has n binary inputs, its truth table will have 2^n possible input combinations.

Consider two inputs A and B. There are two outputs, Sum and Carry. Figure 20 shows the truth table consisting of inputs and outputs. Since there are two inputs, there are four possible combinations.

- When A = 0 and B = 0, adding 0 with 0 yields the sum as zero, with no carry, which is indicated as zero in the truth table.
- When A = 0 and B = 1, adding 0 with 1 yields the sum as 1, with no carry.
- When A = 1 and B = 0, adding 1 with 0 yields the sum as 1, carry as 0.
- When A = 1 and B = 1, adding 1 with 1 will result in 0 and carry 1.

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Figure 20: Truth table for the addition of two bits.

1's Complement

Complementation is one of the very important operations on bits. There are two types of complements: 1's complement and 2's complement.

In 1's complement operation, we flip or invert all the bits in a number; that is, change 0 to 1 and 1 to 0. For example, 1's complement of 1100 is 0011.

2's Complement

2's complement is 1's complement + 1. For example, 2's complement of binary number 01101000 is computed as follows:

Original number → 011010000

1's complement → 100101111

Plus 1 → _____1

2's complement → 100110000

There is a shortcut method. Copy bits from right to left until (and including) the first 1, and then flip all the remaining bits. Figure 21 shows the shortcut method.

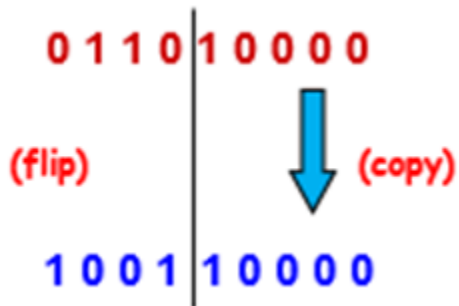


Figure 21: Shortcut method to calculate 2's complement.

Number Representation in a Digital Computer

2.2.1 Unsigned and Signed Magnitude Representation

In programming languages, such as C and Java, variables are declared to support various data types. Data type specifies the type of data variable holds and what is the size requirement. These data types could be integer, floating point, or character type.

There are various ways of representing an integer. The classification of binary integer representation is shown in Figure 1. In unsigned number representation, there are only zero and positive values. If there are n bits in a number, using which we can represent 2^n values. Out of 2^n values, one value represents zero, and the remaining $2^n - 1$ values represent positive numbers. Figure 2 shows the 3-bit binary number combinations and corresponding unsigned values. Since there are three bits, there are eight combinations. The first combination, 000, represents 0, and the remaining combinations represent values from +1 to +7.

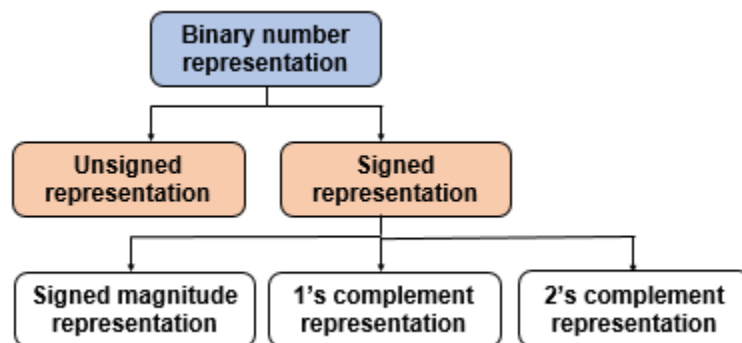


Figure 1: Binary integer representation.

There are three ways to represent negative numbers along with positive numbers:

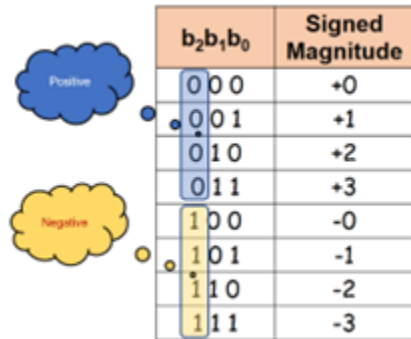
- Signed magnitude representation
- 1's complement representation
- 2's complement representation

$b_2b_1b_0$	Unsigned
000	+0
001	+1
010	+2
011	+3
100	+4
101	+5
110	+6
111	+7

Figure 2: Unsigned binary representation.

In signed magnitude representation, both positive and negative numbers are represented. In this representation, the MSB, that is, the most significant bit, is used to represent the sign of a number. If MSB is 0, then the number is positive. Otherwise, it is negative. Figure 3 shows the signed magnitude representation of 3-bit numbers. With n bits, there are 2^n distinct combinations, out of

which 1 through $2^{n-1}-1$ positive numbers, and $-2^{n-1}-1$ through -1 negative numbers. This leaves two values: one for $+0$ and the other for -0 .



$b_2b_1b_0$	Signed Magnitude
0 0 0	+0
0 0 1	+1
0 1 0	+2
0 1 1	+3
1 0 0	-0
1 0 1	-1
1 1 0	-2
1 1 1	-3

Figure 3: Signed magnitude representation.

2.2.2 Addition of Signed Magnitude Numbers

Three cases of signed magnitude number addition are evaluated to check the suitability of signed magnitude representation for digital computers. While representing the numbers, the table in Figure 3 is referred.

Case 1: Perform $1 + 2$

```

+1 □ 0 0 1
+2 □ 0 1 0
+3 0 1 1

```

The addition result $+3$ in signed magnitude representation is 011 , which matches with the answer, which is obtained after the binary addition. Hence, it is correct.

Case 2: Perform $2 + (-3)$

```

+2 □ 0 1 0
-3 □ 1 1 1
-1 1 0 0 1

```

Ignore the carry generated after binary addition.

When decimal numbers $+2$ and -3 are added, the result obtained is -1 . But the result of binary addition is 001 , which is equal to $+1$. Hence it is incorrect.

Case 3: Perform $3 + (-3)$

```

+3 □ 0 1 1
-3 □ 1 1 1
+0 1 0 1 0
-0

```

When decimal addition of $+3$ and -3 is performed, the result obtained is 0 , which can be represented as either $+0$ or -0 . But the result obtained after adding binary numbers is 010 , which is equal to $+2$. Hence it is incorrect.

Even though the signed magnitude representation is easy to represent positive and negative numbers, it comes with the following two major drawbacks, which makes it unsuitable for computer arithmetic:

- Addition yields incorrect answers for some input combinations.
- There are two representations for 0 . Hence, wastage of one-bit pattern.

2.2.3 1's Complement Representation

1's complement representation is used to represent zero, positive, and negative numbers. The representation of positive numbers is similar to that of signed magnitude representation. That is, the

most significant bit is zero for positive numbers. A negative number is represented by flipping all the bits of the corresponding positive number. For example, +3 is represented as 011. But to represent -3, the following procedure needs to be followed.

Step 1: Consider the magnitude of the negative number and convert it to its binary representation.

The magnitude of -3 is +3, and its representation in binary is 011.

Step 2: Flip all the bits.

So, 1's complement representation of -3 is 100.

Figure 4 shows the 1's complement number representation compared to the signed magnitude representation. Numbers are represented using three bits b_2, b_1 , and b_0 . In 1's complement representation also, there are two representations for 0, that is, +0 and -0. There are three positive numbers and three negative numbers. In both the signed magnitude representation and 1's complement representation, +0 and positive numbers have the same bit pattern. But the representation for -0 and negative numbers are different.

$b_2b_1b_0$	Signed Magnitude	1's Complement
000	+0	+0
001	+1	+1
010	+2	+2
011	+3	+3
100	-0	-3
101	-1	-2
110	-2	-1
111	-3	-0

Figure 4: 3-bit binary representation using signed magnitude and 1's complement representation. For the n -bit number, the number of positive numbers is $2^{n-1}-1$, the number of negative numbers is $2^{n-1}-1$, and there are two representations for 0. The range of numbers that can be represented is given by $-2^{n-1}-1$ to $+2^{n-1}-1$.

2.2.4 Addition of 1's Complement Numbers

Four cases of 1's complement number addition are evaluated to check the suitability of 1's complement representation for digital computers. While representing the numbers, the table in Figure 4 is referred to.

Case 1: Perform $1 + 2$

+1 □ 0 0 1

+2 □ 0 1 0

+3 0 1 1

+3 in binary is 011, which matches the answer obtained after the binary addition. Hence, the addition operation is correct.

Case 2: Perform $2 + (-3)$

+2 □ 0 1 0

-3 □ 1 0 0

-1 1 1 0

When +2 and -3 are added, the result obtained is -1. The result of binary addition is 110, which is also equal to -1. Hence, the addition operation is correct.

Case 3: Perform $3 + (-3)$

+3 □ 0 1 1

-3 □ 1 0 0

0 1 1 1

When decimal addition of +3 and -3 is performed, the result obtained is 0, which can be represented as either +0 or 0. The result obtained after adding binary numbers is 111, which is equal to -0.

Hence, the addition operation is correct.

Case 4: Perform 3 + (-2)

+3 □ 0 1 1

-2 □ 1 0 1

1 1 0 0 0

Ignoring the carry results in 000, which is equal to +0. But when +3-2 is performed, the result obtained is 1, which does not match the result obtained after binary addition. Hence, the addition operation is incorrect. But there is a way to correct this, that is, adding carry to the result obtained.

After adding carry, the result will be 001, which matches the result of decimal addition.

The main advantage of 1's complement representation is that it can be used for arithmetic operations. But whenever there is a final carry generated out of MSB, it needs to be added to the number. The disadvantage is that the representation is not very easy to understand. When there is a carry, it needs to be handled properly to get the right answer.

2.2.5 2's Complement Representation

2's complement representation is used to represent both positive and negative numbers. While representing the positive number, convert the given number to a binary number. Hence, positive number representation is the same for signed magnitude representation, 1's complement representation, and 2's complement representation. For negative number representation, take the magnitude of the number and convert it into binary, and then apply 2's complement operation on the number.

For example, to represent -15 using 8 bits, the following steps are followed.

Step 1: Take the magnitude of -15. The magnitude of -15 is + 15 and then convert + 15 into binary.

+ 15 □ 0 0 0 0 1 1 1 1

Step 2: Find 1's complement of binary and add 1 to it.

1's complement → 1 1 1 1 0 0 0 0

$$\begin{array}{r} + 1 \\ \hline 11110001 \end{array}$$

Figure 5 shows the 3-bit binary representation using signed magnitude, 1's complement representation, and 2's complement representation. It can be observed that the 2's complementation has one representation for 0. Positive number representation is the same as that of other representations. There are four negative numbers. The combination 100, which represents -4, is called a weird number. It is called a weird number because inverting 100 and adding 1 to it results in 100 itself. Hence, this negative number has no positive counterpart. But for all other negative numbers, there is a positive counterpart.

$b_2b_1b_0$	Signed Magnitude	1's Complement	2's Complement
000	+0	+0	+0
001	+1	+1	+1
010	+2	+2	+2
011	+3	+3	+3
100	-0	-3	-4
101	-1	-2	-3
110	-2	-1	-2
111	-3	-0	-1

Figure 5: 3-bit binary number representation using signed magnitude, 1's complement representation, and 2's complement representation.

2.2.6 Addition of 2's Complement Numbers

Four cases of 2's complement number addition are evaluated to check the suitability of 2's complement representation for digital computers. While representing the numbers, the table in Figure 5 is referred to.

Case 1: Perform $1 + 2$

+1 □ 0 0 1

+2 □ 0 1 0

+3 0 1 1

+3 in binary is 011, which matches the answer obtained after the binary addition. Hence, the addition operation is correct.

Case 2: Perform $2 + (-3)$

+2 □ 0 1 0

-3 □ 1 0 1

-1 1 1 1

When +2 and -3 are added, the result obtained is -1. The result of binary addition is 111, which is also equal to -1. Hence, the addition operation is correct.

Case 3: Perform $3 + (-3)$

+3 □ 0 1 1

-3 □ 1 0 1

0 1 0 0 0

When the decimal addition of +3 and -3 is performed, the result obtained is 0, which can be represented as either +0 or 0. The result obtained after adding binary numbers is 1000. Ignoring the carry, we obtain 000, which is equal to +0. Hence, the addition operation is correct.

Case 4: Perform $3 + (-2)$

+3 □ 0 1 1

-2 □ 1 1 0

1 1 0 0 1

When the decimal addition of +3 and -2 is performed, the result obtained is 1. Ignoring the carry results in 001, which is equal to +1. Hence, the addition operation is correct.

All these four cases show that 2's complement representation is suitable for digital computers.

Operation on Bits

2.3.1 Arithmetic Operation: Addition and Subtraction

There are various uses of 2's complement operation. It is mainly used in signed binary number representation and various arithmetic operations on binary numbers, for example, additions, subtractions, etc. Since 2's complement representation is unambiguous, it is employed in computer number representation.

For the addition of two numbers, bitwise addition is performed from LSB to MSB. Sum and carry will be generated per the truth table shown in Figure 1. Subtraction is performed using addition. For example, $A - B$ can be written as $A + (-B)$. That is, $-B$ and A are added where $-B$ is obtained by 2's complement operation.

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Figure 1: Truth table for the addition of two bits.

In the following section, four cases of addition are shown. Assume, $A = 5$ and $B = 4$.

Case 1: $A + B$

+ 5 using 8-bit representation = 0 0 0 0 0 1 0 1

+ 4 using 8-bit representation = 0 0 0 0 0 1 0 0

$A + B$

```
+5→ 00000101
+4→ 00000100
      00001001
```

The decimal equivalent of 0000 1001(2) is +9. When two positive numbers are added, the result should always be positive.

Case 2: $A - B$

$A - B \square A + (-B)$

$\square + 5$ in 8-bit representation = 0 0 0 0 0 1 0 1

$\square - 4$ in 8-bit representation

```
+4→ 00000100
1's complement → 11111011
Add 1 →      1
          11111100
```

$\square A + (-B)$

$$\begin{array}{r}
 00000101 \\
 + \quad 11111100 \\
 \hline
 100000001
 \end{array}$$

Ignore carry. The decimal equivalent of 00000001(2) is +1. Since A is greater than B, the result should be positive.

Case 3: -A + B

□ - 5 in 8-bit representation

$$\begin{array}{l}
 +5 \rightarrow 00000101 \\
 1\text{'s complement} \rightarrow 11111010 \\
 \text{Add 1} \rightarrow \underline{\hspace{1cm}1\hspace{1cm}} \\
 11111011
 \end{array}$$

□ + 4 in 8-bit representation: 0 0 0 0 0 1 0 0

□ -A + B

$$\begin{array}{r}
 11111011 \\
 + \quad 00000100 \\
 \hline
 11111111
 \end{array}$$

The decimal equivalent of 1 1 1 1 1 1 1(2) is -1. The magnitude of -5 is 5, which is more than 4. Hence, the result will have a sign of 5, which is negative in this case.

Case 4: -A - B

-A - B □ - A + (-B)

□ - 5 in 8-bit representation = 1 1 1 1 1 0 1 1

□ - 4 in 8-bit representation = 1 1 1 1 1 1 0 0

□ -A + (-B)

$$\begin{array}{r}
 11111011 \\
 + \quad 11111100 \\
 \hline
 111110111
 \end{array}$$

Ignore carry. The decimal equivalent of 1 1 1 1 0 1 1(2) is -9. Since -5 and -4 are negative numbers, the result will also have a negative sign.

2.3.2. Arithmetic Operation: Sign Extension and Overflow

Sign extension, short known as sext, is the operation in the computer system. Using sign extension, number of bits of the binary number is increased while preserving the value of the number. While adding two binary numbers, care must be taken to ensure that two numbers' sign bits are aligned. Failing which may result in erroneous results.

The process of sign extension is illustrated in Figure 2. The original number 1 0 0 0 1 has five bits. This number needs to be extended to eight bits. Five bits of the original number are copied as it is, and the remaining three bits are filled with the sign of the number. The new number is the sign-extended version of the original number.

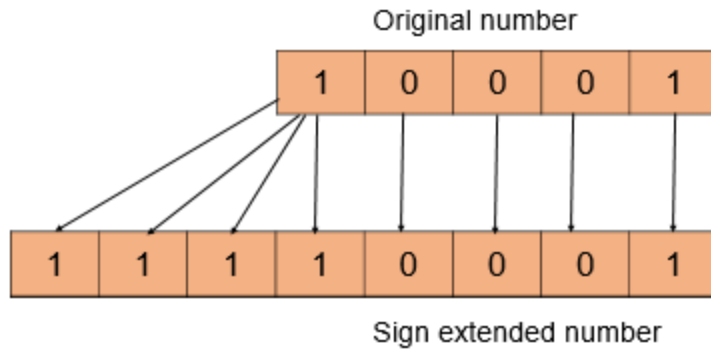


Figure 2: Process of sign extension.

Another issue in computer arithmetic is the overflow condition. Consider two numbers $A = +10$ and $B = +8$. These two numbers are represented using 5-bit 2's complement representation. Adding these two numbers will result in the following.

$A \rightarrow +10 \rightarrow 01010$
 $B \rightarrow +8 \rightarrow 01000$
 $\hline 10010$

The result obtained is -14 instead of +18. The main reason behind this is that it is unable to accommodate +18 using five bits. The range of numbers that can be represented using five bits is -16 to +15. Hence, the result has overflowed the capacity of the representation.

When two 2's complement numbers are added, and they both have the same sign (both positive or both negative), then overflow occurs if and only if the result has the opposite sign. In almost all programming languages, overflow is considered an exception that needs to be handled properly.

2.3.3 Logical Operation: AND, OR, NOT, NAND, NOR, EXOR, and EXNOR

A logic gate is an electronic circuit that acts as a building block for all digital circuits. These circuits perform logical operations. There are three basic logic gates, namely the OR gate, the AND gate, and the NOT gate. These basic logic gates are used to implement the most elementary logic expressions. Other logic gates that are derived from these basic gates are the NAND gate, the NOR gate, the EXCLUSIVE-OR gate, and the EXCLUSIVE-NOR gate.

AND Gate: AND operation is performed by AND gate with two or more inputs and one output. The symbol of the AND gate with two inputs, A and B, and output Y, and the truth table is shown in Figure 3. The output of an AND gate is 1 only when all its inputs are in the 1 state. For all other input combinations, the output is 0. The logic expression is given by

$$Y = A \text{ AND } B$$

$$= A \cdot B$$

$$= AB$$

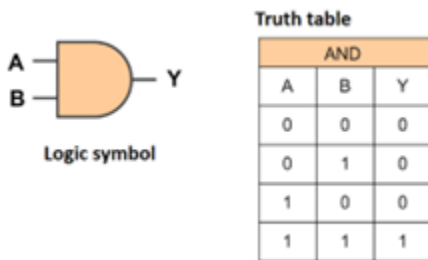


Figure 3: Two-input AND gate and corresponding truth table.

OR Gate: An OR gate performs an ORing operation on the given inputs. The logic gate has two or more inputs and one output. The logic symbol and truth table for the two-input OR gate is shown in Figure 4. The output of an OR gate is 0 only when all its inputs are at 0. For all other possible input combinations, the output is 1. Logic expression for two input OR gate is written as

$$Y = A \text{ OR } B$$

$$= A + B$$

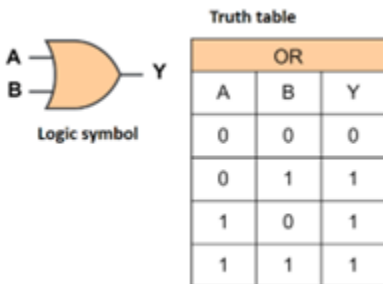


Figure 4: Two-input OR gate and corresponding truth table.

NOT Gate: A NOT gate is a one-input, one-output logic circuit whose output is always the complement of the input. That is, 0 input produces 1 output, and 1 input produces 0 output. The logic symbol and truth table for a two-input OR gate is shown in Figure 5. Logic expression for NOT gate is given by

$$Y = A'$$

$$= A$$

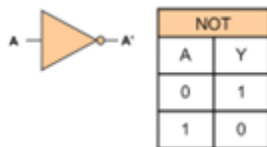


Figure 5: NOT gate and corresponding truth table.

EXOR Gate: The EXCLUSIVE-OR gate, in short, known EX-OR gate, is a two-or-more input, one-output gate. Figure 6 shows the logic symbol and truth table for a two-input EX-OR gate. For a two-input XOR gate, when both the inputs are the same, the output is 0. But when inputs are different, the output is 1. The logic expression of X-OR is given by

$$Y = A \text{ EXOR } B.$$

$$Y = A \oplus B.$$

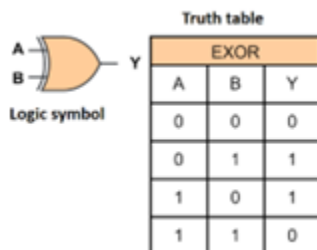


Figure 6: EXOR gate and corresponding truth table.

The output of a multiple-input EX-OR logic function is a logic 1 when the number of 1s in the input sequence is odd. For an even number of 1s, the output will be 0.

NAND Gate: NAND is another logic gate that takes multiple inputs and produces one output. It is equivalent to an AND gate followed by a NOT gate. The truth table for the NAND gate is obtained

from the truth table of an AND gate by complementing the output entries. The output of a NAND gate is a logic 0 when all its inputs are a logic 1. For all other input combinations, the output is a logic 1. Figure 7 shows the logic symbol and truth table for a two-input NAND gate. NAND gate operation is logically expressed as

$$Y = A \text{ NAND } B$$

$$= (AB)'$$

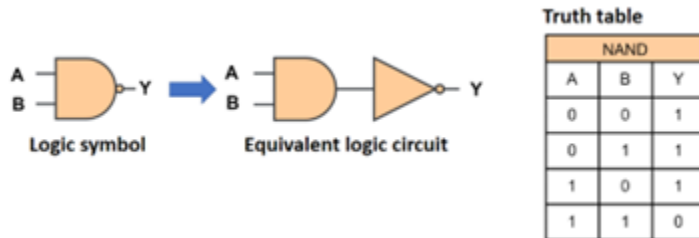


Figure 7: NAND gate and corresponding truth table.

NOR Gate: NOR stands for NOT OR. NOR gate is equivalent to an OR gate followed by a NOT gate. The symbol and truth table of a two-input NOR gate is shown in Figure 8. The output of a NOR gate is a logic 1 when all its inputs are logic "0." For all other input combinations, the output is a logic 0. The output of a two-input NOR gate is logically expressed as

$$Y = A \text{ NOR } B$$

$$= (A + B)'$$

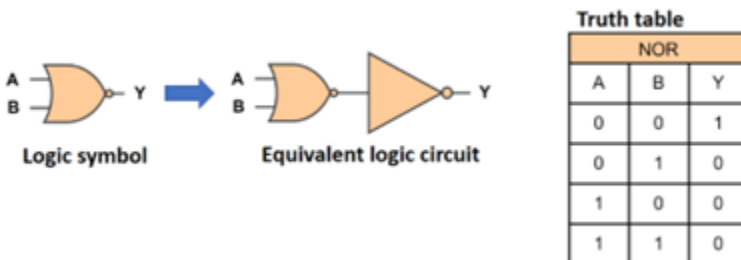


Figure 8: NOR gate and corresponding truth table.

Binary Codes

2.4.1 BCD and ASCII Code

Computer processes the data which is in the form of text, image, number, audio, or video. These are represented, collected, stored, and transmitted using a group of symbols called binary codes. BCD code, ASCII code, Excess-3 Code, and Gray code are some of the important codes used in a computer system.

BCD Code

The full form of BCD is binary coded decimal. In this code, each decimal digit in a number is coded with four binary bits. Hence, this code is also known as 8421 code. Figure 1 shows the binary and BCD equivalent of decimal numbers. Notice that there is no difference between binary and BCD codes for numbers from 0 to 9. The binary equivalent of decimal 10 is 1010, but the same code cannot be used as BCD code. While writing BCD code for decimal 10, each digit is coded separately. That is, the BCD code for 1 is 0001, and for 0, it is 0000.



Decimal	Binary	BCD
0	0000	0000
1	0001	0001
2	0010	0010
3	0011	0011
4	0100	0100
5	0101	0101
6	0110	0110
7	0111	0111
8	1000	1000
9	1001	1001
10	1010	0001 0000

Figure 1: BCD code.

Example: Convert decimal number 345 to BCD. BCD representation of 3 is 0011, 4 is 0100, and 5 is 0101.

345 □ 0011 0100 0101

To convert BCD to decimal, make four bits group from LSB and then convert it to decimal.

Example: Convert BCD number 0111 0101 1001 to decimal. 0111 is 7, 0101 is 5, and 1001 is 9.

0111 0101 1001 □ 759

Most modern computers use byte-oriented memory, in a sense, each location of the memory is capable of storing 1 byte or 8 bits of information. Based on how many digits can be accommodated in each location, there are two types: unpacked BCD and packed BCD. Unpacked and packed BCD representations are shown in Figure 2 and Figure 3, respectively.

In unpacked BCD representation, one BCD digit per byte is used. For example, to represent 48, 2 bytes are needed. Digit 8 is represented using two nibbles. The lower nibble is 1000, and the upper nibble is set to all zeros. Similarly, 4 is represented using two nibbles. The lower nibble is 0100, and the upper nibble is set to all zeros. In both bytes, the upper nibble is redundant. It is as good as representing 0408 in BCD.

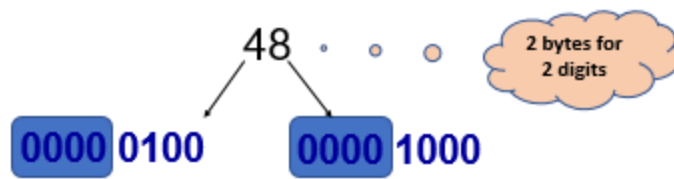


Figure 2: Unpacked BCD representation.

To utilize memory efficiently, packed BCD representation is used. In this method, in one byte of memory, two digits are packed. For example, decimal number 48, where 8 is represented as 1000 and is stored in the lower nibble and 4 is represented as 0100 is stored in the upper nibble.

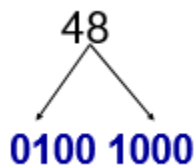


Figure 3: Packed BCD representation.

ASCII Code

Another very popular code that is used in every digital computer and internet is ASCII, i.e., American Standard Code for Information Interchange. This code is developed by American National Standards Institute, ANSI. ASCII provides a character encoding format for text data, which is represented by eight bits. Out of these eight bits, the lower seven bits are used to represent the code, and most significant bit is used as a parity bit. This parity bit is used to indicate a single-bit error in the code. Characters in ASCII encoding include uppercase and lowercase letters A through Z, numerals 0 through 9, and basic punctuation symbols. It also uses some non-printing control characters that were originally intended for use with teletype printing terminals. The ASCII table containing 128 characters is shown in Figure 4.

Dec	Char	Dec	Char	Dec	Char	Dec	Char
0	NUL (null)	32	SPACE	64	@	96	`
1	SOH (start of heading)	33	!	65	A	97	a
2	STX (start of text)	34	"	66	B	98	b
3	ETX (end of text)	35	#	67	C	99	c
4	EOT (end of transmission)	36	\$	68	D	100	d
5	ENQ (enquiry)	37	%	69	E	101	e
6	ACK (acknowledge)	38	&	70	F	102	f
7	BEL (bell)	39	'	71	G	103	g
8	BS (backspace)	40	(	72	H	104	h
9	TAB (horizontal tab)	41	)	73	I	105	i
10	LF (NL line feed, new line)	42	*	74	J	106	j
11	VT (vertical tab)	43	+	75	K	107	k
12	FF (NP form feed, new page)	44	,	76	L	108	l
13	CR (carriage return)	45	-	77	M	109	m
14	SO (shift out)	46	.	78	N	110	n
15	SI (shift in)	47	/	79	O	111	o
16	DLE (data link escape)	48	0	80	P	112	p
17	DC1 (device control 1)	49	1	81	Q	113	q
18	DC2 (device control 2)	50	2	82	R	114	r
19	DC3 (device control 3)	51	3	83	S	115	s
20	DC4 (device control 4)	52	4	84	T	116	t
21	NAK (negative acknowledge)	53	5	85	U	117	u
22	SYN (synchronous idle)	54	6	86	V	118	v
23	ETB (end of trans. block)	55	7	87	W	119	w
24	CAN (cancel)	56	8	88	X	120	x
25	EM (end of medium)	57	9	89	Y	121	y
26	SUB (substitute)	58	:	90	Z	122	z
27	ESC (escape)	59	;	91	[	123	{
28	FS (file separator)	60	<	92	\	124	
29	GS (group separator)	61	=	93	]	125	}
30	RS (record separator)	62	>	94	^	126	~
31	US (unit separator)	63	?	95	_	127	DEL

Figure 4: ASCII Code.

2.4.2 BCD Arithmetic

BCD Addition

Procedure for the addition of two BCD numbers:

- Add the two BCD numbers (binary addition).
- If 4-bit result is equal to or less than 9, then it is a valid BCD number.
- If 4-bit result is greater than 9 or if there is a carry out of the 4-bit then add 6 (0110) to the result.
- If there is a carry generated after addition, then simply add that carry to the next BCD bit group.

Example: Find the sum of two BCD numbers A and B, where:

A = 0011 1001 0101 □ 3 9 5₁₀

B = 0100 0001 0011 □ 4 1 3₁₀

When two BCD numbers are added, the result obtained should be equivalent to decimal number eight hundred and eight. Start from the lower nibble. Add first nibbles. The result obtained is 1000, which is equal to 8 in decimal and is less than 9. Hence, it is a valid BCD number. Add second nibbles, and the result obtained is 1010, which is equal to 10 in decimal. Since 10 is greater than 9, it

is not a valid BCD number. Now, add 0110 to 1010. The answer obtained is 0000 with carry 1. 0000 is a valid BCD number. The carry that is generated should be added to the next nibble. Add the last nibbles with carry, and the result obtained is 1000. 1000 is equal to 8 in decimal and is less than 9. Hence, it is a valid BCD number. The entire process of addition is shown in Figure 5. The final result obtained is 1000 0000 1000, which is equivalent to 808(10).

Find the sum of two BCD numbers A and B, where:

$$\begin{array}{r}
 \\
 A = 0011\ 1001\ 0101 \rightarrow 3\ 9\ 5_{10} \\
 B = \underline{0100\ 0001\ 0011} \rightarrow \underline{4\ 1\ 3}_{10} \\
 1000\ 1010\ 1000 \quad 8\ 0\ 8_{10} \\
 \underline{0110} \\
 \textcircled{1}0000
 \end{array}$$

Figure 5: BCD addition – example.

BCD Subtraction

While performing binary subtraction, 2's complement operation is used. Similarly, in BCD subtraction, 10's complement is used. 10's complement is 9's complement + 1.

Example 1: Obtain 10's complement of BCD number 432.

First, find out 9's complement by subtracting each BCD digit by 9. Then, add 1 to it to get 10's complement.

$$\begin{array}{r}
 999 \\
 (-) \quad \underline{432} \\
 567 \Rightarrow 9\text{'s complement} \\
 (+) \quad \underline{1} \\
 568 \Rightarrow 10\text{'s complement}
 \end{array}$$

Example 2: Perform A - B where A = 475 and B = 239.

BCD representation of A $\Rightarrow$ 0100 0111 0101

BCD representation of B $\Rightarrow$ 0010 0011 1001

Represent -B using 10's complement.

$$\begin{array}{r}
 9\text{'s complement} \quad 999 \Rightarrow 1001\ 1001\ 1001 \\
 \text{BCD number 239} \quad (-) \quad \underline{239} \Rightarrow (-) \underline{0010\ 0011\ 1001} \\
 760 \Rightarrow 0111\ 0110\ 0000 \\
 10\text{'s complement} \quad (+) \quad \underline{1} \Rightarrow \underline{\hspace{1cm}}1 \\
 761 \Rightarrow 0111\ 0110\ 0001
 \end{array}$$

Now add A and -B.

A $\Rightarrow$ 0100 0111 0101

-B $\Rightarrow$ 0111 0110 0001

0010 0011 0110 $\Rightarrow$ 236